



Wrocław
University
of Science
and Technology

IBM AI Racing League Country Challenge 2026

Building an autonomous AI driver for the TORCS
simulator.

AiBC team: Wiktoria, Óscar, Mario, Krystian, Jakub, Dominik, Diego, Álvaro

Date: 16/06/2026



HR EXCELLENCE IN RESEARCH



Wrocław
University
of Science
and Technology



HR EXCELLENCE IN RESEARCH

Agenda

What we will cover

01 The Challenge

IBM competition brief, the Corkscrew circuit, our constraints

02 Our Approach

Why deterministic control over deep RL and the key design decision

03 System Architecture

5-layer pipeline from Docker to Optuna optimiser

04 AlphaDriver

Control logic, steering formula, speed caps

05 Results

112.58 s best lap, 29.9% improvement, 10/10 reliability

06 Business Relevance

Sensor-to-action patterns in e-commerce and logistics

07 Lessons & Next Steps

What we learned and where we go from here



Wrocław
University
of Science
and Technology



HR EXCELLENCE IN RESEARCH

The Challenge

What IBM asked us to do

Competition Brief

Event: IBM AI Racing League, Country Challenge 2026

Objective: Build an autonomous Python AI driver for the TORCS simulator

Track: Corkscrew circuit, high-speed sweeping corners, elevation changes and variable-radius turns

Evaluation: Lap time, track adherence and quality of the technical strategy presentation

The Rules

- Python-only modifications, **no touching the C/C++ simulator** source
- Must run entirely inside Docker/Podman container, **no external APIs at runtime**
- IBM Granite AI certification required for all team members

Our Constraints

Team: 8 students - one semester, alongside other coursework

Hardware: Standard laptops - no GPU cluster, no cloud compute





Wrocław
University
of Science
and Technology



HR EXCELLENCE IN RESEARCH

The Basis - Problem Statement

What we had?

The Challenge

Build an autonomous AI driver in Python for the TORCS racing simulator, using only raw sensor data every 20 ms to output steering, acceleration, braking and gear commands.

The Inputs

19 laser-range track-edge distances ($\pm 45^\circ$), speed in 3 axes, angular velocity, track position, wheel spin velocities, RPM, gear, fuel, damage and lap time.

The Constraints

No C/C++ modifications allowed. Must run inside Docker. IBM Granite designated as mandatory AI engine. Full team IBM SkillsBuild certification required.

Baseline lap time on Day 1: **160.59 s** | Action space: **steer · accel · brake · gear** | Environment: **TORCS Corkscrew track**



Wrocław
University
of Science
and Technology



HR EXCELLENCE IN RESEARCH

Our Approach

Deterministic control over deep RL

The Options We Considered

Deep RL (PPO/SAC): Learn a neural policy from scratch via reward feedback

Imitation learning: Clone behaviour from expert demonstrations

Deterministic control: Hand-craft a sensor-to-action rule set, tune with Bayesian search

Why We Chose Deterministic + Optuna

- Deep RL needs thousands of laps to converge and we had one semester
- Deterministic policies are fully inspectable, debuggable and reproducible
- Optuna's TPE Bayesian search gives the best of both worlds: principled optimisation without a training loop
- A known-good baseline (snakeoil3 reference client) gave us a fast starting point
- Result: went from baseline to competitive in days, not weeks

The Trade-off We Accepted

- No in-lap adaptation, the policy does not recover from near-wall states
- Parameters tuned for Corkscrew specifically, not yet generalised across other tracks



Wrocław
University
of Science
and Technology



HR EXCELLENCE IN RESEARCH

Experimental AI Integration

Local LLM Control

Pipeline Architecture

UDP-to-HTTP Communication Bridge: Intercepted real-time telemetry from the `snakeoil3` client and routed it to a local Ollama API instance.

State Vector Serialization: Formatted 41-feature continuous variables (track sensors, speeds, RPM) into high-density JSON prompts at every control cycle.

Parsing & Runtime Fallback Logic: Enforced strict JSON schema validation for target actions. Implemented automatic state retention in case of payload corruption or timeout.

Performance & Constraints

Spatial Awareness Achieved

The trained model successfully learned the circuit topology, completing full laps without off-track violations.

The 20 ms Latency Bottleneck

Inference token generation on quantized models consistently breached the simulator's strict 20 ms execution frame.

Conservative Driving Profile

Compounding input lag required an overly restrictive speed profile to maintain tracking stability, yielding uncompetitive lap times.

Core Engineering Insight & System Dependency

The generative control loop proved functionally viable but was limited by immutable stock vehicle physics. Had the competition guidelines permitted physical modifications - such as adjusting tire friction coefficients or expanding aerodynamic downforce to mitigate delayed response parameters - a local LLM architecture could achieve highly competitive baselines.



Wrocław
University
of Science
and Technology

System Architecture

5-layer pipeline, fully containerised

The Pipeline

1. **Simulator:** TORCS runs headlessly in Docker; GUI navigated automatically via `wmctrl + xte`
2. **Communication:** snakeoil3 UDP client on port 3001, sensor packet in and action packet out every 20 ms
3. **Sensor Processing:** SensorProcessor normalises 41 raw fields, computes clearance, slip ratio and boolean flags
4. **Control Policy:** AlphaDriver reads angle, trackPos and 19 track sensors → outputs steer, accel, brake, gear
5. **Optimisation:** improver.py wraps the lap loop as an Optuna objective; TPE search updates `best_params.json`

Key Design Choices

Telemetry: 57-column CSV log written every 20 ms, used to detect bugs and guide optimisation

CI/CD: GitHub Actions runs a full lap on every push to main; results committed to `history.csv`

Config: All tunable constants are in `best_params.json`, separated from code and updated without a redeploy



HR EXCELLENCE IN RESEARCH



Wrocław
University
of Science
and Technology



HR EXCELLENCE IN RESEARCH

Implementation - The files

What have we achieved?

snakeoil3_gym.py

UDP socket client - handles TORCS SCR handshake, packet parsing and response serialisation.

sensor_processor.py

Parses 19 track sensors + speed + RPM into typed SensorObservation. Logs 57-column CSV per step.

car_agent.py

Deterministic PD steering + dual-signal corner anticipation. Loads best_params.json at startup.

improver.py

Optuna TPE Bayesian search over $\pm 10\%$ local parameter space. Minimises mean lap time across 5 laps per trial.

TORCS Sensor -> SensorProcessor -> AlphaDriver -> Optuna -> best_params.json

112.58s

Best lap time achieved

29.9%

Improvement over baseline

10/10

Lap completion rate

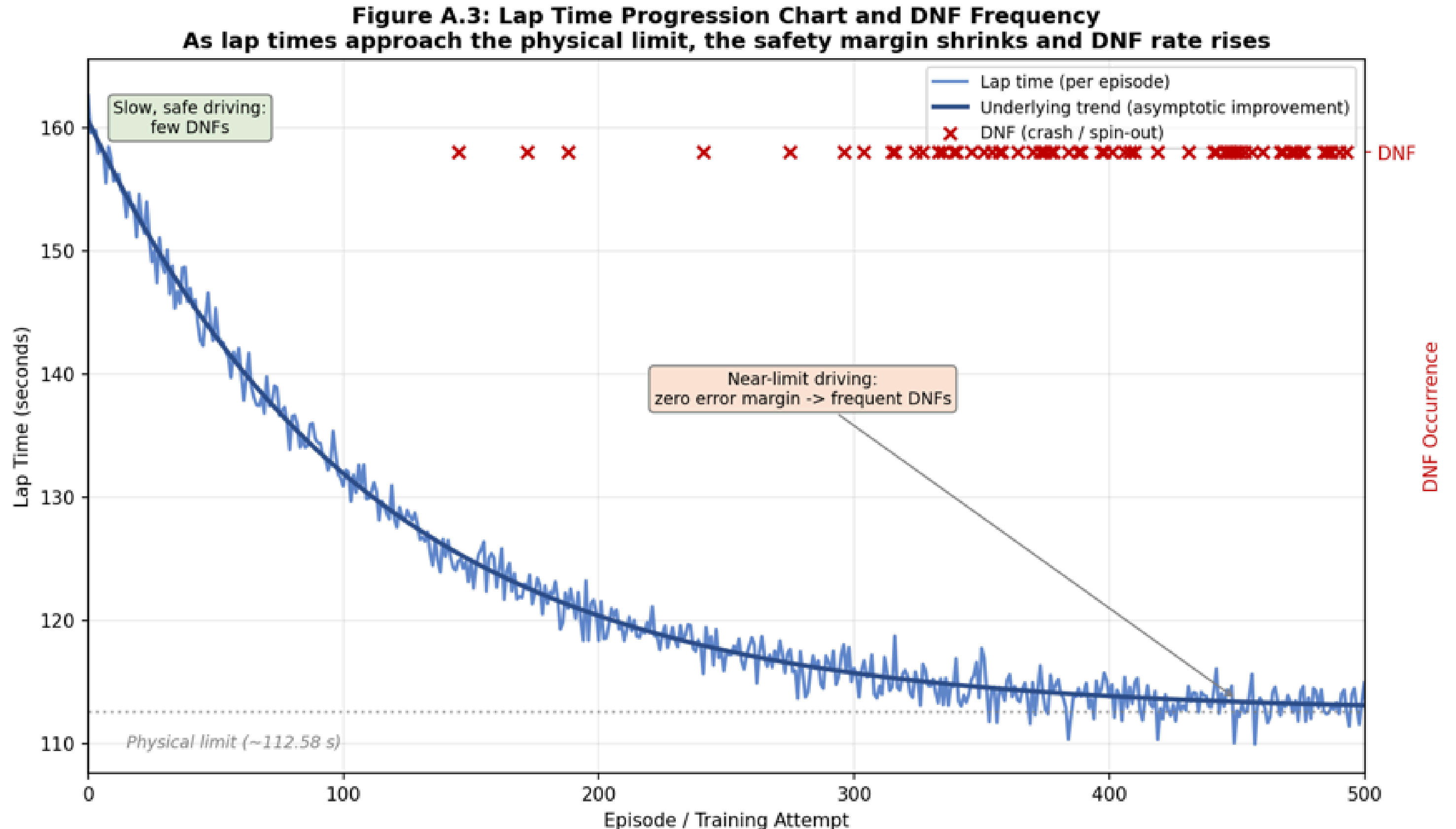


Wrocław
University
of Science
and Technology



HR EXCELLENCE IN RESEARCH

Implementation - The graph





Implementation - The code

```
1 # -- Steering (reference PD formula) -----
2 steer = self._steer(angle, track_pos)
3 r['steer'] = steer
4
5 # -- Corner anticipation: forward clearance cap -----
6 track = np.array(s['track'], dtype=np.float32)
7 safe = np.where(track < 0, 200.0, track) # replace -1 (3
8 fwd_p75 = float(np.percentile(safe[5:14], 75)) # 75th percent
9
10 fwd_cap = float(np.interp(fwd_p75,
11     p.get('CORNER_DISTs', [5, 15, 30, 50, 80, 120, 200]),
12     p.get('CORNER_SPEEDS', [40, 55, 80, 110, 150, 200, 280])
13 ))
14
15 # -- Corner anticipation: left/right asymmetry cap -----
16 left_avg = float(np.mean(safe[0:4]))
17 right_avg = float(np.mean(safe[15:19]))
18 corner_asym = abs(left_avg - right_avg)
19
20 asym_cap = float(np.interp(corner_asym,
21     p.get('ASYM_BREAKS', [0, 15, 35, 60, 90, 130]),
22     p.get('ASYM_SPEEDS', [280, 220, 160, 110, 70, 50])
23 ))
24
25 corner_target = min(fwd_cap, asym_cap, float(p['TARGET_SPEED']))
26
27 # -- Steer-based speed reduction -----
28 steer_target = float(p['TARGET_SPEED']) - abs(steer) * float(p['STEER_SPEED_FACTOR'])
29 target = min(corner_target, max(40.0, steer_target))
30
31 # -- Throttle / Brake -----
32 if speed < target:
33     r['accel'] = 1.0 if (target - speed) > 5.0 else min(1.0, (target - speed) / 5.0 + 0.5)
34     r['brake'] = 0.0
35 else:
36     r['accel'] = 0.0
37     r['brake'] = min(1.0, (speed - target) / 50.0)
```

```
1 def objective(trial) -> float:
2     params = _make_params(trial) # builds full param dict with local search
3     driver = AlphaDriver(params=params)
4     lap_times = []
5     for lap_idx in range(_cfg['laps']):
6         client = connect_torcs(port=_cfg['port'])
7         lap_t, dist = run_lap(client, driver, optimal_time=_best_score)
8         lap_times.append(lap_t if lap_t > 0 else DNF_PENALTY)
9         if lap_t == 0:
10             while len(lap_times) < _cfg['laps']:
11                 lap_times.append(DNF_PENALTY)
12             break
13         _navigate_menu(first_run=False) # reset car to start
14     score = float(np.mean(lap_times))
15     if score < _best_score and all(t < DNF_PENALTY for t in lap_times):
16         # persist new best immediately
17         out = dict(params)
18         out['best_lap_time'] = score
19         with open(_PARAMS_FILE, 'w') as f:
20             json.dump(out, f, indent=2)
21     return score
```



Wrocław
University
of Science
and Technology

Implementation - The run



112.58s



Automatic tests

Github Actions pipeline

The Pipeline

After every commit that changes something in our implementation, the GitHub Actions pipeline starts. The pipeline runs a simulation of the drive without a graphic interface and then saves the results in a table. With this approach we know immediately what exact change made simulations worse than before.

main	github-actions[b...	All time
Commits on Jun 15, 2026	results: run 27539583571 reward=620858.91 lap=112.58s github-actions[bot] committed 20 hours ago	458f618
Commits on Jun 14, 2026	results: run 27511702698 reward=620858.91 lap=112.58s github-actions[bot] committed yesterday	00fb79c
Commits on May 11, 2026	results: run 25686586711 reward=620858.91 lap=112.58s github-actions[bot] committed on May 11	9e98be9
Commits on May 7, 2026	results: run 25520810091 reward=620649.67 lap=114.16s github-actions[bot] committed on May 7	3a51a0c
	results: run 25499390782 reward=619911.43 lap=115.23s github-actions[bot] committed on May 7	8bb2009
	results: run 25476746068 reward=620484.69 lap=116.87s github-actions[bot] committed on May 7	53c77d1
	results: run 25464811270 reward=619868.23 lap=120.77s github-actions[bot] committed on May 7	c73fa11



Wrocław
University
of Science
and Technology



HR EXCELLENCE IN RESEARCH

Knowledge gained?

What have we learned?

1. Containerise from Day 1

Docker-first approach eliminated environment chaos across 8 machines. Native installs waste days; containers take minutes. This discipline now applies to every future deployment project.

2. Decouple Parameters from Code

Storing all tunable constants in `best_params.json` (not hardcoded) meant improving the car required zero code changes - just run Optuna and commit. A powerful MLOps pattern.

3. Instrument Early, Analyse Often

57-column per-step CSV telemetry from day one surfaced the steer sign bug and drove the asymmetry-based corner anticipation idea. Earlier logging = faster iteration.

4. Local Search Beats Brute Force

Wide-range Optuna exploration produced DNFs. $\pm 10\%$ local search around a known baseline rapidly compressed lap time from 130 s $\rightarrow$ 112.58 s. Targeted search outperforms random exploration.

CI is non-negotiable · Auto lap on every push to main caught regressions in minutes · `history.csv` = objective, timestamped proof of progress